COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/ Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/07/02026	**DATE SUBMITTED:**	07=15-2026
NAME:	EUNICE E. MORADILLO	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 1___ Basic C++ Programming

1. PROCEDURE OUTPUT

SECTIONS	SAMPLE CODE
Header File Declaration Section	`#include <iostream>>`
Global Declaration Section	none

Class Declaration and Method Definition Section	`void showSum(double a, double b);`
	`bool isGreater(double a, double b);`
	`bool logicOps(bool a, bool b);`

Main Function	`int main() {`
	`  double a, b;`
	`  std::cout << "Enter two numbers: ";`
	`  std::cin >> a >> b;`
	
	`  showSum(a, b);`
	`  std::cout << a << (isGreater(a, b) ? " is" : " is NOT") << " greater than " << b << ".\n";`
	`  logicOps(true, false);`
	
	`  return 0;`
	`}`

Method Definition	`void showSum(double a, double b) {`
	`  std::cout << "Sum: " << a + b << "\n";`
	`}`
	
	`bool isGreater(double a, double b) {`
	`  return a > b;`
	`}`
	
	`bool logicOps(bool a, bool b) {`
	`  std::cout << "AND: " << (a && b) << " OR: " << (a || b) << " NOT A: " << !a << " NOT B: " << !b << "\n";`
	`  return true;`
	`}`

Table 1-1. C++ Structure Code for Answer

```cpp
#include <iostream>>

// camelCase
//snake_
//PascalCase

class Triangle{
    private:
        double totalAngle, angleA, angleB, angleC;
    public:
        //constructor
        Triangle(double A, double B, double C);
        //set a different
        void setAngles(double A, double B, double C);
        const bool validateTriangle();

};

int main(){

    Triangle set1(40, 50,110);
    if(set1.validateTriangle()){
        std::cout<<"the shape is a valid triangle \n.";
    }else{
        std::cout<< "the shape is not a valid triangle \n.";
    }
    return 0;
}

Triangle::Triangle (double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

void Triangle::setAngles(double A, double B, double C){
    angleA = A;
    angleB = B;
    angleC = C;
    totalAngle = A + B + C;
}

const bool Triangle::validateTriangle(){
    return (totalAngle <= 180 );
}
```

Code snippet

```
the shape is not a valid triangle
```

Output snippet

OBSERVATIONS AND COMMENTS

The triangle class correctly checks whether the three sets of angles form a triangle. In the code, I used 40, 50 and 110, that sums up an angle of 200. Since the sum of the angles are greater, which is 200 over a triangle which has 180, the validateTriangle() returns false and is considered as not a valid triangle.

Table 1-2. ILO B output observations and comments.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SUPPLEMENTARY 1

```cpp
#include <iostream>

bool swapNumbers(double &a, double &b);

int main() {
    double x = 6, y = 7;

    std::cout << "Before swap: x = " << x << ", y = " << y << "\n";
    bool success = swapNumbers(x, y);
    std::cout << "After swap:  x = " << x << ", y = " << y << "\n";
    std::cout << "Swap successful: " << success << "\n";

    return 0;
}

bool swapNumbers(double &a, double &b) {
    double temp = a;
    a = b;
    b = temp;
    return true;
}
```

CODE SNIPPET

```
Before swap: x = 6, y = 7
After swap:  x = 7, y = 6
Swap successful: 1
```

OUTPUT SNIPPET

ANALYSIS:

This code shows the function directly changes and swaps the variables instead of copies inside the function swapNum. Using the temp variable, it is used to hold one value before the swap happens because it might overwrite the number or not swap. In main(), the user will write two numbers and them swapNums(x,y) will be called and it will print.

SUPPLEMENTARY 2

```cpp
#include <iostream>

double kelvinToFahrenheit(double kelvinValue);
double fahrenheitToKelvin(double fahrenheitValue);

int main() {
    int choice;

    std::cout << "(1) Fahrenheit to Kelvin\n";
    std::cout << "(2) Kelvin to Fahrenheit\n";
    std::cout << "Enter your choice (1 or 2): ";
    std::cin >> choice;

    if (choice == 1) {
        double fahrenheitValue;
        std::cout << "Enter temperature in Fahrenheit: ";
        std::cin >> fahrenheitValue;
        double kelvinValue = fahrenheitToKelvin(fahrenheitValue);
        std::cout << fahrenheitValue << " F is equal to " << kelvinValue << " K.\n";
    } else if (choice == 2) {
        double kelvinValue;
        std::cout << "Enter temperature in Kelvin: ";
        std::cin >> kelvinValue;
        double fahrenheitValue = kelvinToFahrenheit(kelvinValue);
        std::cout << kelvinValue << " K is equal to " << fahrenheitValue << " F.\n";
    } else {
        std::cout << "Invalid choice. Please enter 1 or 2.\n";
    }

    return 0;
}

double kelvinToFahrenheit(double kelvinValue) {
    return (kelvinValue - 273.15) * 9.0 / 5.0 + 32.0;
}

double fahrenheitToKelvin(double fahrenheitValue) {
    return (fahrenheitValue - 32.0) * 5.0 / 9.0 + 273.15;
}
```

CODE SNIPPET

```
(1) Fahrenheit to Kelvin
(2) Kelvin to Fahrenheit
Enter your choice (1 or 2): 1
Enter temperature in Fahrenheit: 45
45 F is equal to 280.372 K.
```

OUTPUT SNIPPET

ANALYSIS:

This code converts temperature from kelvin to fahrenheit and fahrenheit to kelvin using the temperature input by the user. The function uses a double and returns a double so it can do decimal numbers because temperatures can't be whole numbers usually. After the user type in a temperature value, it will get passed into the function then convert it then print.

SUPPLEMENTARY 3

```cpp
#include <iostream>

double mySqrt(double num) {
    if (num == 0) return 0;
    double guess = num;
    for (int i = 0; i < 20; i++) {
        guess = (guess + num / guess) / 2;
    }
    return guess;
}

double distance(double x1, double y1, double x2, double y2) {
    double dx = x2 - x1;
    double dy = y2 - y1;
    return mySqrt(dx * dx + dy * dy);
}

int main() {
    double x1, y1, x2, y2;
    std::cout << "Enter x1 y1 x2 y2: ";
    std::cin >> x1 >> y1 >> x2 >> y2;
    std::cout << "Distance: " << distance(x1, y1, x2, y2) << "\n";
    return 0;
}
```

CODE SNIPPET

```
Enter x1 y1 x2 y2: x1
Distance: 0
```

OUTPUT SNIPPET

ANALYSIS

This code calculates the distance between two points. mySqrt() is a function that guesses the square root, then keeps improving that guess in a loop until it's very close to correct. The distance() function finds how far apart the points are on the x-axis and y-axis, squares those differences, adds them, then sends that number to mySqrt() to get the final distance.

SUPPLEMENTARY 4

```cpp
#include <iostream>
#include <string>

double mySqrt(double num) {
    if (num == 0) return 0;
    double guess = num;
    for (int i = 0; i < 20; i++) {
        guess = (guess + num / guess) / 2;
    }
    return guess;
}

class Triangle {
private:
    double angleA, angleB, angleC, sideA, sideB, sideC;
public:
    Triangle(double A, double B, double C, double sA, double sB, double sC)
        : angleA(A), angleB(B), angleC(C), sideA(sA), sideB(sB), sideC(sC) {}

    bool validateTriangle() { return angleA + angleB + angleC <= 180; }

    double computePerimeter() { return sideA + sideB + sideC; }

    double computeArea() {
        double s = computePerimeter() / 2;
        return mySqrt(s * (s - sideA) * (s - sideB) * (s - sideC));
    }

    std::string classifyTriangle() {
        if (angleA > 90 || angleB > 90 || angleC > 90) return "Obtuse-angled";
        if (angleA == 90 || angleB == 90 || angleC == 90) return "Others (Right-angled)";
        return "Acute-angled";
    }
};

int main() {
    Triangle set1(40, 30, 110, 9, 12, 15);

    if (set1.validateTriangle()) {
        std::cout << "Valid triangle.\n";
        std::cout << "Perimeter: " << set1.computePerimeter() << "\n";
        std::cout << "Area: " << set1.computeArea() << "\n";
        std::cout << "Classification: " << set1.classifyTriangle() << "\n";
    } else {
        std::cout << "The shape is not a valid triangle.\n";
    }
    return 0;
}
```

CODE SNIPPET

```
Valid triangle.
Perimeter: 36
Area: 54
Classification: Obtuse-angled
```

OUTPUT SNIPPET

ANALYSIS

This code used a skeleton from the ILO B and added perimeter, area and classification.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: I use Gemini to search and helped with the concept explanation

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I only used gemini to help me generate the right wordings in my analysis of my code idea

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.